

전자전기컴퓨터설계실험Ⅱ

LAB 1. VS Code Verilog

환경설정 매뉴얼

이번 매뉴얼의 목표

- 전가산기 하나로 코드 편집부터 파형 해석까지 확인한다.
- Windows와 Vivado 2026.1을 준비한다.
- VS Code·Git 설치 → 도구 경로 확인 → 예제 열기
- 언어 서버 확인 → 8개 입력 시뮬레이션 → 파형 해석

확인: 파일을 여는 데서 끝내지 않고, 회로의 결과를 설명한다.

도구별 역할

도구	역할
VS Code	소스 편집, 터미널과 작업 실행
Git / GitHub CLI	소스 이력 관리 / GitHub 작업(사용 시)
vscode-slang	언어 서버와 연결하여 코드 분석 기능 제공
slang-server	정의 이동, 심벌 정보, 문법 오류·경고 진단
Vivado XSim	HDL 컴파일과 회로 시뮬레이션
VaporView	생성된 VCD 파일을 파형으로 표시

확인: VS Code에서 XSim을 실행하고 VaporView로 결과를 본다.

1. VS Code와 Git 설치

- VS Code Windows 설치 안내에서 설치 프로그램을 받는다.
- 개인 PC에서는 User Setup을 사용하고 PATH 추가 옵션을 확인한다.
- Git for Windows를 설치한다. 명령줄에서 Git을 사용하는 옵션을 선택한다.
- 설치 후 VS Code와 기존 터미널을 닫고 다시 연다.
- Terminal → New Terminal에서 PowerShell을 연다.

공식 안내: VS Code Windows 설치

1. 버전과 실행 경로 확인

```
git --version  
code --version  
Get-Command git, code | Select-Object Name, Source
```

- 버전과 실행 파일 위치가 출력되어야 한다.
- PATH는 명령 이름으로 실행 파일을 찾는 경로 목록이다.
- 명령을 찾지 못하면 설치·PATH를 확인하고 VS Code를 다시 실행한다.

확인: VS Code는 버전 외에 커밋·아키텍처도 출력할 수 있다.

1. GitHub CLI를 사용하는 경우

GitHub CLI는 Git과 별도로 설치한다.

```
gh --version  
Get-Command gh | Select-Object Name, Source  
gh auth login  
gh auth status
```

- 로그인 안내에서 GitHub.com과 브라우저 인증을 선택한다.
- 이 예제의 git clone과 시뮬레이션에는 필수가 아니다.

공식 안내: gh auth login

2. Vivado의 bin 폴더를 PATH에 추가

- Vivado 설치 폴더에서 아래 세 파일이 있는 bin 폴더를 찾는다.
- xvlog.bat / xelab.bat / xsim.bat
- 계정의 환경 변수 편집 → 사용자 변수 Path → 편집 → 새로 만들기
- 기존 항목을 유지하고, 본인의 실제 bin 폴더를 추가한다.

```
C:\AMDDesignTools\2026.1\Vivado\bin
```

확인: 위 경로는 예시다. 등록 후 VS Code까지 종료하고 다시 연다.

2. Vivado 시뮬레이션 도구 확인

```
Get-Command xvlog.bat, xelab.bat, xsim.bat  
xvlog -version  
xelab -version  
xsim -version
```

- 세 도구가 모두 2026.1로 실행되는지 확인한다.
- Git·VS Code는 --version, Vivado는 -version이다.
- GUI 실행 성공과 터미널 명령의 경로 설정은 각각 확인한다.



2. PATH를 등록했는데도 실행되지 않는다면

- 이미 열려 있던 PowerShell은 변경 전 PATH를 사용한다.
- 기존 PowerShell과 VS Code의 모든 창을 종료한다.
- 시작 메뉴에서 PowerShell을 새로 열어 아래 명령을 확인한다.
- 이전 PowerShell에서 code를 실행하면 VS Code도 이전 환경을 물려받을 수 있다.

```
Get-Command xvlog.bat, xelab.bat, xsim.bat
```

확인: 녹화에서는 오래 열어둔 PowerShell 대신 새로 실행하니 정상 동작했다.

3. GitHub에서 예제 받기

예제를 저장할 상위 폴더에서 다음 명령을 실행한다.

```
git clone --branch v1.0.2 --depth 1 https://github.com/Glaysia/ece2_26_2_2.git  
cd ece2_26_2_2
```

- v1.0.2 태그로 고정된 수업 자료와 예제를 내려받는다.
- 마지막 줄은 내려받은 저장소 폴더로 이동한다.
- 태그로 받으면 나오는 detached HEAD 안내는 오류가 아니다.

수업 예제: GitHub 저장소 열기

3. 수업 워크스페이스 열기

저장소 루트에서 실행한다.

```
code ece2_26_2_2.code-workspace
```

- GUI: File → Open Workspace from File...에서 저장소 루트의 ece2_26_2_2.code-workspace를 선택한다.
- HDL 프로젝트별 워크스페이스 파일은 필요하지 않다.
- 수업 예제임을 확인하고 작업 영역 신뢰를 선택한다.

확인: 탐색기의 fpga_projects_hdl 아래에서 example_full_adder_hdl을 펼친다.

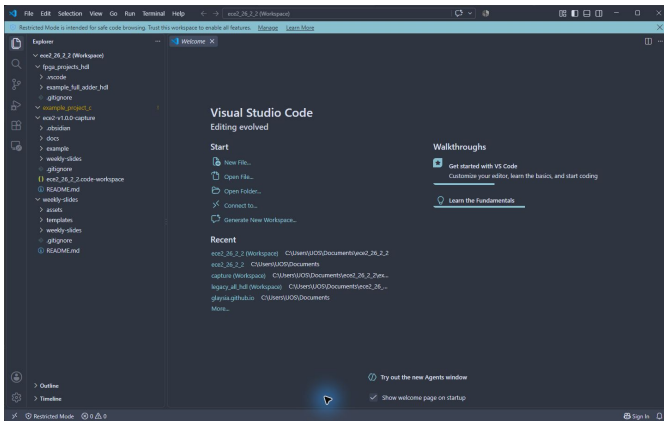
3. 워크스페이스 신뢰하기

- 처음 열 때 작성자를 신뢰하는지 묻는 창이 나타날 수 있다.
- 수업 GitHub 저장소에서 받은 자료인지 확인한다.
- 확인한 수업 자료라면 Yes, I trust the authors를 선택한다.
- 상위 폴더 전체를 신뢰하는 옵션은 선택할 필요가 없다.
- 이미 제한 모드라면 상단 Manage 또는 명령 팔레트의 Workspaces: Manage Workspace Trust에서 상태를 확인한다.

확인: Restricted Mode에서는 작업 실행과 일부 확장 기능이 제한된다.

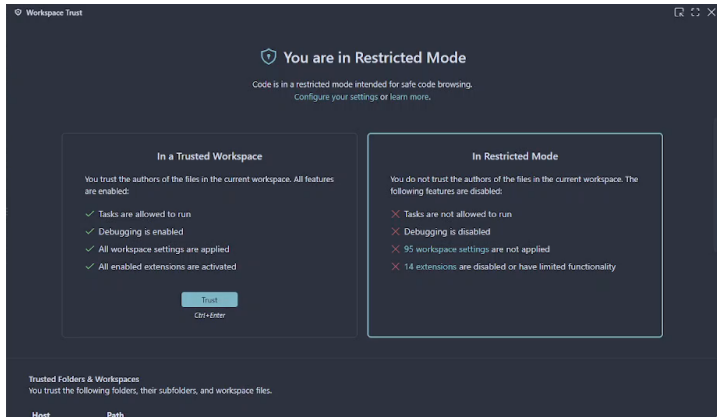


3. 실제 화면 : 제한 모드 확인



상단 안내와 왼쪽 아래 Restricted Mode를 확인한다.

3. 실제 화면 : Trust 선택



수업 저장소임을 확인한 뒤 왼쪽 Trust를 선택한다.



3. 예제 파일의 역할

경로	역할
src/half_adder.v	반가산기
src/full_adder.v	반가산기 두 개로 구성한 전가산기
sim/tb_full_adder.sv	8개 입력을 자동 검사하는 테스트벤치
.slang/server.json, sources.f	언어 서버 설정과 HDL 소스 목록
../.vscode/tasks.json	상위 HDL 폴더의 공용 작업 설정
scripts/	실행 스크립트
build/	실행 후 생성되는 파형과 로그

4. 확장 두 개 설치

Ctrl+Shift+X → Extensions에서 확장 ID로 검색한다.

```
Hudson-River-Trading.vscode-slang  
lramseyer.vaporview
```

- vscode-slang: Verilog/SystemVerilog 언어 지원
- VaporView: 시뮬레이션 파형 보기

확인: 이름이 비슷한 확장과 구분하여 두 확장을 설치한다.

4. 명령으로 확장 설치·확인

```
code --install-extension Hudson-River-Trading.vscode-slang  
code --install-extension lramseyer.vaporview  
code --list-extensions --show-versions |  
    Select-String 'slang|vaporview'
```

확인: 확장 ID와 설치 버전이 출력된다.

설치 화면에서 설치 여부를 확인해도 된다.



4. Install slang-server 알림

- src/full_adder.v를 연다.
- 설치 알림에서 Install slang-server를 선택한다.
- 서버 다운로드와 초기 분석이 끝날 때까지 기다린다.
- 필요하면 Ctrl+Shift+P → Developer: Reload Window를 실행한다.
- 이미 서버가 설치된 PC에서는 알림이 다시 나오지 않을 수 있다.

확인: 자동 설치에서는 서버를 PATH에 추가하거나 slang.path를 지정하지 않는다.

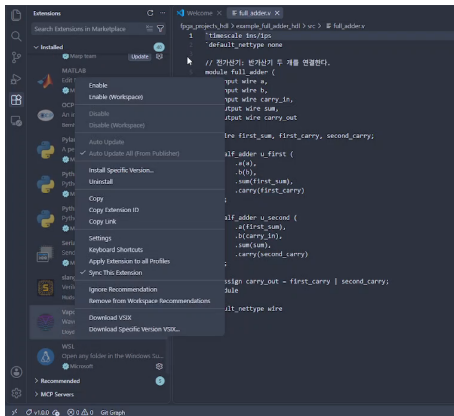
공식 안내: 서버 자동 설치 · 알림 문구는 버전에 따라 다를 수 있음

4. 설치된 확장도 활성화 여부 확인

- Ctrl+Shift+X에서 slang과 VaporView를 각각 찾는다.
- 비활성화돼 있다면 톱니바퀴 메뉴의 Enable (Workspace)를 선택한다.
- 이미 활성화돼 있다면 다시 설정할 필요가 없다.
- 재시작 안내가 나오면 따르고, 필요하면 Developer: Reload Window를 실행한다.
- .v 파일의 오른쪽 아래가 Plain Text가 아닌 Verilog인지 확인한다.

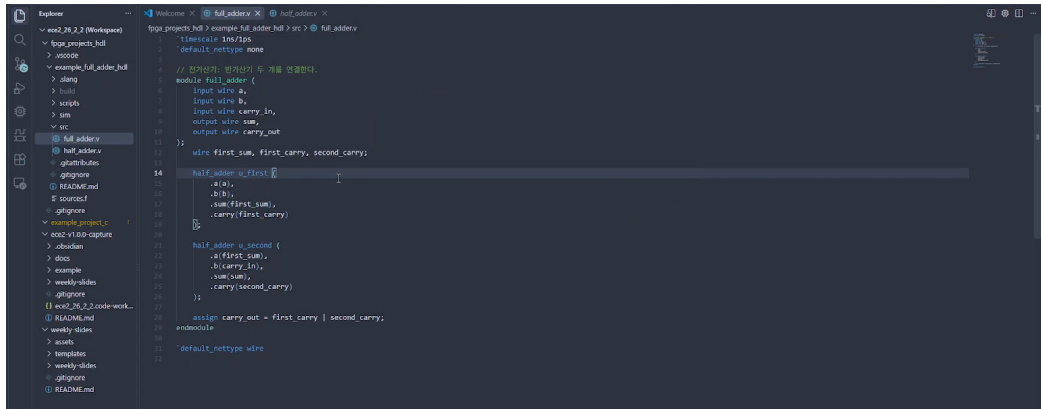
확인: 워크스페이스 신뢰와 개별 확장의 활성화는 따로 확인한다.

4. 실제 화면 : Enable (Workspace)



확장을 설치했어도 현재 워크스페이스에서 꺼져 있으면 활성화한다.

5. 실제 화면 : 전가산기 코드



```
1 timescale 1ns/1ps
2 `default_nettype none
3
4 // 전가산기: 반가산기 두 개를 연결한다.
5 module full_adder (
6     input wire a,
7     input wire b,
8     input wire carry_in,
9     output wire sum,
10    output wire carry_out
11);
12    wire first_sum, first_carry, second_carry;
13
14    half_adder u_first (
15        .a(a),
16        .b(b),
17        .sum(first_sum),
18        .carry(first_carry)
19    );
20
21    half_adder u_second (
22        .a(first_sum),
23        .b(carry_in),
24        .sum(sum),
25        .carry(second_carry)
26    );
27
28    assign carry_out = first_carry | second_carry;
29 endmodule
30
31 `default_nettype wire
32
```

반가산기 두 개의 연결을 확인한다. 색상 표시만으로 분석 성공을 판단하지 않는다.



5. 언어 서버가 필요한 이유

- 모듈·신호·연결 관계를 분석하여 여러 파일의 회로를 읽도록 돕는다.
- 색상과 심벌 정보로 코드의 구조를 구분한다.
- Ctrl+클릭으로 사용한 모듈의 정의에 바로 이동한다.
- 문법 오류와 경고를 편집 중 확인한다.

확인: 색상만 확인하지 말고 정의 이동과 오류 진단까지 시험한다.

실제 색상은 선택한 VS Code 테마에 따라 달라진다.

5. Ctrl+클릭으로 반가산기 열기

full_adder.v에서 half_adder 위에 Ctrl+클릭 또는 F12.

```
half_adder u_first (  
    .a(a),  
    .b(b),  
    .sum(first_sum),  
    .carry(first_carry)  
);
```

확인: half_adder.v의 모듈 정의로 이동한다.

Alt+왼쪽 화살표로 돌아온다. 마우스를 올려 심벌 정보도 확인한다.



5. 문법 오류를 확인하고 복원하기

```
assign carry_out = first_carry | second_carry;
```

- 마지막 세미콜론을 잠시 지우고 저장한다.
- 오류 밑줄과 Problems(Ctrl+Shift+M)의 진단을 읽는다.
- Ctrl+Z로 복원하고 다시 저장한다.
- 오류가 사라진 것을 확인한 뒤 시뮬레이션한다.

오류 위치가 다음 줄에 표시될 수도 있다. 경고도 내용을 읽고 원인을 확인한다.

5. 분석할 소스 목록

sources.f에 세 소스가 등록되어 있다.

```
src/half_adder.v  
src/full_adder.v  
sim/tb_full_adder.sv
```

- .slang/server.json은 이 목록을 사용한다.
- 새 파일을 추가할 때 언어 서버의 분석 대상도 확인한다.
- 다른 실험의 같은 이름 모듈이 섞이지 않도록 프로젝트 범위를 확인한다.

공식 안내: 언어 서버 프로젝트 설정



6. 전가산기의 예상 결과

a	b	carry_in	carry_out	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

입력 3개의 합을 두 출력으로 표현한다.

$\{\text{carry_out}, \text{sum}\}$
 $= a + b + \text{carry_in}$

반가산기 두 개와 OR 연산을 연결한다.

조합회로이므로 클록 입력이 없다.



6. 테스트벤치는 무엇을 검사하는가?

- `sim/tb_full_adder.sv`가 입력 000부터 111까지 인가한다.
- 각 입력을 10 ns 동안 유지한다.
- 입력 변경 1 ns 뒤 출력과 예상값을 비교한다.
- 8개 입력을 모두 검사하고 총 80 ns에 종료한다.

확인: 1 ns는 시뮬레이션의 검사 대기 시간이다.
실제 FPGA의 전파 지연을 측정한 값은 아니다.

7. VS Code 작업으로 실행하기

- 소스 파일을 모두 저장한다.
- Ctrl+Shift+P → Tasks: Run Task → HDL: Check tools
- Vivado 도구 세 개의 경로와 버전을 확인한다.
- Ctrl+Shift+B → HDL: Simulate full adder
- 또는 Tasks: Run Task에서 같은 시뮬레이션 작업을 선택한다.

확인: 작업이 없으면 수업 워크스페이스와 작업 설정의 등록 위치를 확인한다.

7. 성공 결과 확인

PASS: all 8 full-adder input combinations

- build/full_adder.vcd가 생성된다.
- 터미널의 Waveform:은 파형 위치를 알려 준다.
- Logs:는 이번 실행의 로그 폴더를 알려 준다.
- PASS가 없으면 실패한 단계의 로그부터 확인한다.

확인: 단순히 실행이 끝난 것과 8개 검사를 통과한 것을 구분한다.

7. 터미널에서 직접 실행할 때

저장소 루트의 PowerShell에서 예제 폴더로 이동한 뒤 실행한다.

```
cd example/fpga_projects_hdl/example_full_adder_hdl  
powershell.exe -NoProfile -ExecutionPolicy Bypass `  
-File scripts/check-tools.ps1  
powershell.exe -NoProfile -ExecutionPolicy Bypass `  
-File scripts/simulate.ps1
```

줄 끝의 백틱은 PowerShell의 줄 연결 기호다. 뒤에 공백을 넣지 않는다.

실행 정책 옵션은 해당 프로세스에 적용된다. 영구 실행 정책 변경은 필요 없다.

8. VaporView에서 VCD 열기

- build/full_adder.vcd를 연다.
- 텍스트로 열리면 우클릭 → Open With... → VaporView
- VaporView Netlist에서 tb_full_adder를 펼친다.
- a, b, carry_in, sum, carry_out을 더블클릭하거나 +로 추가한다.
- 파형 창에서 Zoom to Fit(Ctrl+0)으로 전체 구간을 본다.

공식 안내: VaporView 신호 추가와 파형 조작

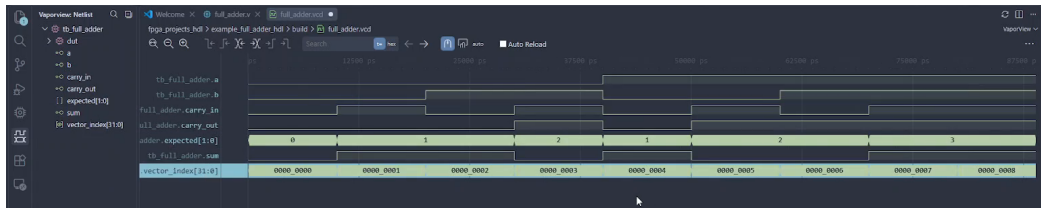
8. 시간축과 진리표 비교

시간	a, b, carry_in	carry_out, sum
0-10 ns	000	00
30-40 ns	011	10
70-80 ns	111	11

- 세 구간을 먼저 찾고 나머지 다섯 구간도 확인한다.
- 커서를 각 구간에 놓고 입력과 출력을 함께 읽는다.
- 시간축이 ps이면 $10,000 \text{ ps} = 10 \text{ ns}$ 이다.

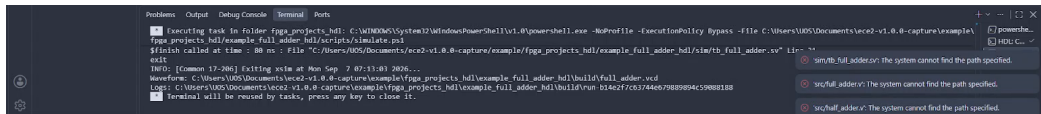
확인: 그림을 저장하기 전에 8개 입력이 진리표와 일치하는지 확인한다.

8. 실제 화면 : 8개 입력의 파형



시간 단위는 ps이다. 10,000 ps마다 입력이 바뀌며 80 ns까지 비교한다.

8. 실제 화면 : 실행 완료와 별도 오류 구분



- 터미널에 80 ns 종료와 생성된 VCD 경로가 표시된다.
- 오른쪽 소스 경로 오류 알림은 파형 생성 결과와 구분한다.
- 소스 분석·이동 문제가 남으면 언어 서버 설정의 상대경로 기준을 확인한다.

8. 수정한 코드의 결과 다시 확인

- 저장 → 시뮬레이션 재실행 → 파형 다시 열기 또는 새로고침
- 이번 실행의 PASS와 새 VCD를 확인한다.
- 실패 후 이미 열려 있던 파형을 새 결과로 해석하지 않는다.

예제 스크립트는 실행 시작 시 이전 대표 VCD를 제거한다.
모든 검사가 성공한 경우 새 결과를 대표 VCD로 복사한다.

9. 실험 전 보고서에 담을 것

- 설계 내용과 핵심 코드
- 시험 입력과 예상 결과
- VS Code에서 확인한 파형과 결과 해석
- 파일명·신호명·시간축·입출력 변화가 읽히는 스크린샷

확인: 전가산기로 환경을 확인한 뒤 LAB1 전체 10개 실험에 적용한다.

이 배포 프로젝트에는 전가산기 예제만 들어 있다.

9. 파형을 설명하는 문장

30–40 ns 구간에서 $a=0$, $b=1$, $\text{carry_in}=1$ 이다.

세 입력의 합은 2이므로 $\text{carry_out}=1$, $\text{sum}=0$ 을 예상한다.

파형에서도 같은 값이 확인되어 이 입력 조건의 결과가 진리표와 일치한다.

확인: 시험 조건 → 예상 결과 → 관찰 결과 순서로 설명한다.

9. LAB1~LAB3 제출·시연 방식

- 실험 전: 해당 LAB 전체의 VS Code 시뮬레이션과 해석
- 당일: 조교가 무작위로 2개 선정 → 보드 시연 → 출석부 기록
- 실험 후: 전체 Vivado 시뮬레이션과 선정된 2개의 시연 사진
- 사진에는 실험 번호·입력 조건·확인한 동작을 설명한다.

수업 운영의 기준은 OT TeX를 따른다. LAB1 10개 · LAB2 8개 · LAB3 7개.

10. 도구 경로가 잘못되었을 때

```
Get-Command xvlog.bat, xelab.bat, xsim.bat -All  
$env:VIVADO_BIN
```

- 실제로 호출되는 버전과 경로 순서를 확인한다.
- PATH 수정 전부터 켜 둔 PowerShell과 VS Code를 모두 종료하고 새로 실행한다.
- VIVADO_BIN이 설정되어 있으면 스크립트가 그 경로를 사용할 수 있다.

개별 실행에서 실제 설치 경로를 지정하는 방법:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `  
-File scripts/simulate.ps1 `  
-VivadoBin 'C:\AMDDesignTools\2026.1\Vivado\bin'
```



10. 언어 서버가 동작하지 않을 때

- 설치 알림이 없으면 먼저 F12로 기존 설치 여부를 확인한다.
- 확장 활성화, 작업 영역 신뢰와 열린 프로젝트 폴더를 확인한다.
- 오른쪽 아래 언어 모드: .v는 Verilog, .sv는 SystemVerilog
- sources.f에 필요한 파일이 등록되었는지 확인한다.
- 다운로드 실패는 Output의 slang 관련 로그와 네트워크를 확인한다.

확인: 오류 메시지를 그대로 확인하고 조교에게 전달한다.

10. 시뮬레이션·파형 문제 확인

증상	확인할 것
작업 메뉴가 없음	수업 워크스페이스와 작업 설정의 등록 위치를 확인했는가?
컴파일 실패	compile.txt의 첫 오류, 세미콜론 복원 여부
모듈 연결 실패	elaborate.txt의 최상위·모듈 연결 오류
PASS가 없음	simulation.txt의 실패 입력·예상값·실제값
VCD가 없음	이번 실행이 성공했는가?
파형이 비어 있음	신호 추가, 올바른 VCD, Zoom to Fit

로그 위치: build/run-.../

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.